

Statistical and Machine Learning SS26

Solutions – Exercise Sheet 1

Exercise 1: My First R Script

```
# a) Current working directory
getwd()

# b) Set working directory
setwd("path/to/SML_SS26")
```

Exercise 2: Object Assignment

```
# a)
x <- 42
y <- 8
x + y; x - y; x * y; x / y

# b)
my_name <- "Mara"
print(my_name)
mode(my_name) # returns "character"

# c)
n1 + n2 # Error - R is case-sensitive, so N2 and n2 are different objects
```

Exercise 3: Data Types

```
# use either
# mode() : reflects storage type; how R stores data internally; numeric, list
# class() : reflects how R methods treat it - more specific; integer, factor, data.frame, matrix
a <- 3.14      # numeric (double)
b <- 7L        # integer
c <- "Statistics" # character
d <- TRUE      # logical
```

```
e <- factor(c("low", "medium", "high", "medium", "low")) # factor

# a) 7L vs 7
b2 <- 7
is.integer(b); is.integer(b2) # TRUE / FALSE
is.numeric(b); is.numeric(b2) # TRUE / TRUE
# 'L' signals R to store object as an integer (no decimal),
# 7 stores a double (7.000...); both are "numeric"

# b) Factor e
levels(e) # "high" "low" "medium"
length(e) # 5
as.integer(e) # encodes factor levels as integers (alphabetical order: high=1, low=2, medium=3)
```

Exercise 4: Vectors

```
# a)
v1 <- c(4, 8, 15, 16, 23, 42)
v2 <- c("Berlin", "Belgrade", "Basel", "Barcelona")
v3 <- c(TRUE, FALSE, TRUE, TRUE, FALSE)

# b)
sum(v1); mean(v1); median(v1); var(v1); sd(v1)
min(v1); max(v1)

# c)
1:20
seq(0, 1, by = 0.1)
rep(c(1, 2, 3), times = 4)

# d) vectorised operations - predicted output:
x <- c(1, 2, 3, 4); y <- c(10, 20, 30, 40)
x + y # 11 22 33 44
x * y # 10 40 90 160
x^2   # 1 4 9 16
```

Exercise 5: Indexing and Subsetting

```
v1 <- c(4, 8, 15, 16, 23, 42)

# a) third element
v1[3] # 15
```

```

# b) all except second
v1[-2]

# c) elements > 15
v1[v1 > 15]

# d) replace fourth element
v1[4] <- 99

# e)
v1[c(1, 3, 5)] # elements at positions 1, 3, 5

```

Exercise 6: Matrices

```

# a) by column (default) vs by row
M <- matrix(1:12, nrow = 4, ncol = 3) # by column
M

```

```

##      [,1] [,2] [,3]
## [1,]    1    5    9
## [2,]    2    6   10
## [3,]    3    7   11
## [4,]    4    8   12

```

```

M_row <- matrix(1:12, nrow = 4, ncol = 3, byrow = TRUE)
M_row

```

```

##      [,1] [,2] [,3]
## [1,]    1    2    3
## [2,]    4    5    6
## [3,]    7    8    9
## [4,]   10   11   12

```

```

# b) names
colnames(M) <- c("A", "B", "C")
rownames(M) <- c("r1", "r2", "r3", "r4")

```

```

# c) access
M[3, 2] # row 3, col 2

```

```

## [1] 7

```

```

M[2, ] # entire row 2

```

```

##  A B C
##  2 6 10

```

```
M[, 3]      # entire col 3
```

```
## r1 r2 r3 r4  
##  9 10 11 12
```

```
M[1:2, 2:3] # sub-matrix
```

```
##      B  C  
## r1 5   9  
## r2 6  10
```

```
# d) apply  
apply(M, 1, sum)  # row (1) sums
```

```
## r1 r2 r3 r4  
## 15 18 21 24
```

```
apply(M, 2, mean) # column (2) means
```

```
##      A      B      C  
## 2.5  6.5 10.5
```

```
# e) element-wise vs matrix product  
A <- matrix(c(1,2,3,4), 2, 2)  
B <- matrix(c(5,6,7,8), 2, 2)  
A * B      # element-wise multiplication
```

```
##      [,1] [,2]  
## [1,]    5  21  
## [2,]   12  32
```

```
A %*% B      # proper matrix multiplication (dot products)
```

```
##      [,1] [,2]  
## [1,]   23  31  
## [2,]   34  46
```

Exercise 7: Arrays

```
data(UCBAdmissions)  
str(UCBAdmissions)  
  
# a) admitted vs rejected  
apply(UCBAdmissions, 1, sum)
```

```

# b) males vs females
apply(UCBAdmissions, 2, sum)

# c) by department
apply(UCBAdmissions, 3, sum)

# d) admitted/rejected by gender
apply(UCBAdmissions, c(1, 2), sum)

# e) males/females by department
apply(UCBAdmissions, c(2, 3), sum)

```

Exercise 8: Lists

```

# a)
student <- list(
  name    = "Anna",
  age     = 23,
  courses = c("Statistics", "Econometrics"),
  passed  = TRUE
)

# b) three access methods
student["courses"]    # returns list
student[["courses"]]  # returns character vector
student$courses        # returns character vector
class(student["courses"]) # "list"
class(student[["courses"]]) # "character"

# c) add element
student$grade <- 1.7

# d) structure
str(student)

```

Exercise 9: Data Frames

```

# a)
students_df <- data.frame(
  name  = c("Paolo", "Ben", "Clara", "Tarik", "Yan"),
  age   = c(22, 24, 23, 25, 21),
  grade = c(1.3, 2.7, 4.0, 1.7, 5.0)
)

```

```
students_df$passed <- students_df$grade <= 4.0
```

```
# b)
str(students_df)
```

```
## 'data.frame': 5 obs. of 4 variables:
## $ name : chr "Paolo" "Ben" "Clara" "Tarik" ...
## $ age : num 22 24 23 25 21
## $ grade : num 1.3 2.7 4 1.7 5
## $ passed: logi TRUE TRUE TRUE TRUE FALSE
```

```
summary(students_df)
```

```
##      name      age      grade      passed
## Length:5      Min.   :21      Min.   :1.30      Mode :logical
## Class :character 1st Qu.:22      1st Qu.:1.70      FALSE:1
## Mode :character  Median :23      Median :2.70      TRUE :4
##                  Mean   :23      Mean   :2.94
##                  3rd Qu.:24      3rd Qu.:4.00
##                  Max.   :25      Max.   :5.00
```

```
print(students_df)
```

```
##      name age grade passed
## 1 Paolo  22   1.3   TRUE
## 2 Ben    24   2.7   TRUE
## 3 Clara  23   4.0   TRUE
## 4 Tarik  25   1.7   TRUE
## 5 Yan    21   5.0  FALSE
```

```
# c)
students_df$grade
```

```
## [1] 1.3 2.7 4.0 1.7 5.0
```

```
students_df[, "grade"]
```

```
## [1] 1.3 2.7 4.0 1.7 5.0
```

```
students_df[3,]
```

```
##      name age grade passed
## 3 Clara  23     4   TRUE
```

```
students_df[2, 3]
```

```
## [1] 2.7
```

```
# d) grade_letter
students_df$grade_letter <- ifelse(students_df$grade <= 1.5, "A",
                                   ifelse(students_df$grade <= 2.5, "B",
                                           ifelse(students_df$grade <= 3.5, "C",
                                                   ifelse(students_df$grade <= 4.0, "D", "F"))))
```

```
# e) iris
data(iris)
nrow(iris); ncol(iris)      # 150 rows, 5 columns
colnames(iris)
table(iris$Species)        # 50 per species
subset(iris, Sepal.Length > 5 & Species == "setosa")
```

Exercise 10: Functions

```
# a)
celsius_to_fahrenheit <- function(temp_c) {
  9/5 * temp_c + 32
}
celsius_to_fahrenheit(c(0, 100, -40)) # 32, 212, -40

# b)
my_summary <- function(x) {
  list(mean = mean(x), sd = sd(x), min = min(x), max = max(x), n = length(x))
}
v1 <- c(4, 8, 15, 16, 23, 42)
my_summary(v1)
v2 <- c(4, 8, 15, 16, 23, 42, NA)
my_summary(v2) # function doesn't work, output: NAs

# c) with na.rm
my_summary <- function(x, na.rm = FALSE) {
  list(
    mean = mean(x, na.rm = na.rm),
    sd   = sd(x,   na.rm = na.rm),
    min  = min(x,  na.rm = na.rm),
    max  = max(x,  na.rm = na.rm),
    n    = sum(!is.na(x)) # count non-missing values (!is.na returns TRUE/FALSE, sum() counts TRUES)
  )
}
my_summary(c(1, 2, NA, 4, 5), na.rm = TRUE) # NAs ignored
my_summary(c(1, 2, NA, 4, 5), na.rm = FALSE) # NAs kept, output is NA
```

Exercise 11: Control Flow

```
# a) if-else
grade <- 2.3
if (grade <= 4.0) print("Pass") else print("Fail")
grade <- 4.7
if (grade <= 4.0) print("Pass") else print("Fail")

# b) vectorised
ifelse(students_df$grade <= 4.0, "Pass", "Fail")

# c) for loop
for (i in 1:10) print(i^2)

# d) while loop
val <- 1
while (val <= 1000) val <- val * 2
print(val) # 1024

# e) sapply vs for loop
sapply(c(-20, 0, 20, 37, 100), celsius_to_fahrenheit)

results <- c()
for (temp in c(-20, 0, 20, 37, 100)) {
  results <- c(results, celsius_to_fahrenheit(temp))
}
results
# both produce the same output
# sapply() is preferred in R (shorter, more readable, faster for large vectors).
# For loop requires manually managing the results vector.
```

Exercise 12: Reading and Writing Data

```
# a)
dax.prices <- read.csv("dax_prices.csv")

# b)
head(dax.prices)
class(dax.prices$DAX)
#alternatively:
is.numeric(dax.prices$DAX) # Yes, the prices are interpreted correctly

# c)
colnames(dax.prices)[colnames(dax.prices) == "DAX"] <- "DAX Prices"
#alternatively:
names(dax.prices)[2] <- "DAX Prices" # because DAX is the second column
```



```
# d)
write.table(dax.prices, "dax_prices.txt", sep = ";", dec = ",", row.names = FALSE)

# e)
dax.prices.txt <- read.table("dax_prices.txt", header = TRUE, sep = ";", dec = ",")
class(dax.prices.txt$DAX)
```

Exercise 13: Linear Regression

```
data(swiss)
str(swiss)      # 47 observations, 6 variables
```

```
## 'data.frame':   47 obs. of  6 variables:
## $ Fertility      : num  80.2 83.1 92.5 85.8 76.9 76.1 83.8 92.4 82.4 82.9 ...
## $ Agriculture    : num  17 45.1 39.7 36.5 43.5 35.3 70.2 67.8 53.3 45.2 ...
## $ Examination    : int   15 6 5 12 17 9 16 14 12 16 ...
## $ Education      : int   12 9 5 7 15 7 7 8 7 13 ...
## $ Catholic       : num   9.96 84.84 93.4 33.77 5.16 ...
## $ Infant.Mortality: num  22.2 22.2 20.2 20.3 20.6 26.6 23.6 24.9 21 24.4 ...
```

```
summary(swiss)
```

```
##      Fertility      Agriculture      Examination      Education
## Min.   :35.00   Min.   : 1.20   Min.   : 3.00   Min.   : 1.00
## 1st Qu.:64.70   1st Qu.:35.90   1st Qu.:12.00   1st Qu.: 6.00
## Median :70.40   Median :54.10   Median :16.00   Median : 8.00
## Mean   :70.14   Mean   :50.66   Mean   :16.49   Mean   :10.98
## 3rd Qu.:78.45   3rd Qu.:67.65   3rd Qu.:22.00   3rd Qu.:12.00
## Max.   :92.50   Max.   :89.70   Max.   :37.00   Max.   :53.00
##      Catholic      Infant.Mortality
## Min.   : 2.150   Min.   :10.80
## 1st Qu.: 5.195   1st Qu.:18.15
## Median :15.140   Median :20.00
## Mean   :41.144   Mean   :19.94
## 3rd Qu.:93.125   3rd Qu.:21.70
## Max.   :100.000   Max.   :26.60
```

```
# b) simple regression
fit_simple <- lm(Fertility ~ Education, data = swiss)
summary(fit_simple)
```

```
##
## Call:
## lm(formula = Fertility ~ Education, data = swiss)
##
## Residuals:
```

```
##      Min      1Q  Median      3Q      Max
## -17.036  -6.711  -1.011   9.526  19.689
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  79.6101     2.1041  37.836 < 2e-16 ***
## Education   -0.8624     0.1448  -5.954 3.66e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 9.446 on 45 degrees of freedom
## Multiple R-squared:  0.4406, Adjusted R-squared:  0.4282
## F-statistic: 35.45 on 1 and 45 DF,  p-value: 3.659e-07
```

```
# Significant effect (p < 0.01)
# each additional % in education is associated with ~0.86 fewer fertility points
```

```
# c) multiple regression
fit_multi <- lm(Fertility ~ Education + Infant.Mortality, data = swiss)
summary(fit_multi)
```

```
##
## Call:
## lm(formula = Fertility ~ Education + Infant.Mortality, data = swiss)
##
## Residuals:
##      Min      1Q  Median      3Q      Max
## -15.3906  -6.0088  -0.9624   5.8808  21.0736
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   48.8213     8.8904   5.491 1.88e-06 ***
## Education     -0.8167     0.1298  -6.289 1.27e-07 ***
## Infant.Mortality 1.5187     0.4287   3.543 0.000951 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 8.426 on 44 degrees of freedom
## Multiple R-squared:  0.5648, Adjusted R-squared:  0.545
## F-statistic: 28.55 on 2 and 44 DF,  p-value: 1.126e-08
```

```
# R² increases compared to simple model
fitted(fit_multi)
```

```
##      Courtelary      Delemont Franches-Mnt      Moutier      Neuveville      Porrentruy
##      72.73695      75.18692      75.41611      73.93467      67.85703      83.50260
##      Broye      Glane      Gruyere      Sarine      Veveyse      Aigle
##      78.94644      80.10412      74.99777      75.26147      81.12994      64.08025
##      Aubonne      Avenches      Cossonay      Echallens      Grandson      Lausanne
##      72.11221      73.49631      73.13803      79.38480      72.66239      56.63299
##      La Vallee      Lavaux      Morges      Moudon      Nyon      Orbe
##      48.89029      71.84574      67.99164      80.39061      64.38399      67.15773
##      Oron      Payerne Paysd'enhaut      Rolle      Vevey      Yverdon
```

```
##      79.89771      78.43353      73.70824      65.40982      65.04601      76.45919
##      Conthey      Entremont      Herens      Martigwy      Monthey      St Maurice
##      70.12062      73.99197      74.98052      73.38448      77.04943      68.50456
##      Sierre      Sion      Boudry La Chauxdfnd      Le Locle      Neuchatel
##      71.12642      65.69354      69.85138      70.97178      66.90852      57.61878
##      Val de Ruz ValdeTravers V. De Geneve      Rive Droite      Rive Gauche
##      73.47905      72.71969      32.87538      52.77890      54.44949
```

```
resid(fit_multi)
```

```
##      Courtelary      Delemont Franches-Mnt      Moutier      Neuveville      Porrentruy
##      7.46305247      7.91308062      17.08388945      11.86533212      9.04297468      -7.40259744
##      Broye      Glane      Gruyere      Sarine      Veveyse      Aigle
##      4.85355949      12.29588210      7.40222883      7.63852800      5.97005513      0.01975064
##      Aubonne      Avenches      Cossonay      Echallens      Grandson      Lausanne
##      -5.21220511      -4.59630702      -11.43803208      -11.08480137      -0.96239491      -0.93299307
##      La Vallee      Lavaux      Morges      Moudon      Nyone      Orbe
##      5.40970706      -6.74573763      -2.49164239      -15.39060686      -7.78399316      -9.75773028
##      Oron      Payerne Paysd'enhaut      Rolle      Vevey      Yverdon
##      -7.39771486      -4.23352702      -1.70824336      -4.90982013      -6.74601189      -11.05919235
##      Conthey      Entremont      Herens      Martigwy      Monthey      St Maurice
##      5.37938439      -4.69196568      2.31948366      -2.88447809      2.35057489      -3.50455588
##      Sierre      Sion      Boudry La Chauxdfnd      Le Locle      Neuchatel
##      21.07357890      13.60645756      0.54861853      -5.27178255      5.79148238      6.78122292
##      Val de Ruz ValdeTravers V. De Geneve      Rive Droite      Rive Gauche
##      4.12094781      -5.11969270      2.12462073      -8.07889783      -11.64948871
```

```
# d) coefficients and vcov
```

```
coef(fit_multi)
```

```
##      (Intercept)      Education Infant.Mortality
##      48.8212736      -0.8166573      1.5187190
```

```
vcov(fit_multi)
```

```
##      (Intercept)      Education Infant.Mortality
## (Intercept)      79.0397411 -0.295353323      -3.725024810
## Education      -0.2953533      0.016860451      0.005528234
## Infant.Mortality -3.7250248      0.005528234      0.183744369
```

```
# diagonal of vcov = variances of the estimates = SE2
```

```
# e) interaction term
```

```
fit_interaction <- lm(Fertility ~ Education * Infant.Mortality, data = swiss)
summary(fit_interaction)
```

```
##
## Call:
## lm(formula = Fertility ~ Education * Infant.Mortality, data = swiss)
##
## Residuals:
```

```
##      Min      1Q  Median      3Q      Max
## -14.255  -5.733  -1.955   6.361  19.882
##
## Coefficients:
##                      Estimate Std. Error t value Pr(>|t|)
## (Intercept)          58.90098   15.29718   3.850 0.000387 ***
## Education            -1.70641    1.10438  -1.545 0.129641
## Infant.Mortality       0.99724    0.77350   1.289 0.204199
## Education:Infant.Mortality 0.04666    0.05751   0.811 0.421641
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 8.459 on 43 degrees of freedom
## Multiple R-squared:  0.5713, Adjusted R-squared:  0.5414
## F-statistic: 19.1 on 3 and 43 DF,  p-value: 5.035e-08

# p-value of Education:Infant.Mortality > 0.05, the interaction is not significant
```